



**Objective & Lecture Mapping:** In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

## Part 1: Practical Implementation — Building the Search Agent

### Step 1.1: Exposing the World Model

Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week\_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:  

```
'grid_size': (self.width, self.height)
'walls': list(self.walls)
'all_food': list(self.food_positions)
```

### Step 1.2: Implementing BFS, DFS, and UCS

You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent` .
2. Import required structures: `from collections import deque` and `import heapq` .
3. **BFS:** Create `def bfs_search(...)` . Use a FIFO queue ( `deque.popleft()` ). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)` . Use a LIFO stack ( `list.pop()` ). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)` . Use a Priority Queue ( `heapq.heappop()` ) ordered by the total path cost  $g(n)$ .
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

### Step 1.3: Executing and Comparing Offline Plans

The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__` , add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'` .
2. In `sense_and_act(self, percept)` , check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']` , execute the search method matching `self.active_algo` , and store the resulting sequence of actions in `self.plan` .
4. Return the first action from the plan using `return self.plan.pop(0)` .
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

## Part 2: Theoretical Evaluation

Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

State Space : The set of all possible states reachable from the initial state, connected by actions.

Search Tree: shows the different paths the agent explores to reach the goal and can have multiple paths to the same state.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

It prevents the agent from revisiting the same states that has been explored, which prevent the agent from falling into a infinite loop.  
If a DFS agent run without the reached set, it might get stuck in an infinite loop.

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

BFS explores the path level by level, therefore it is guaranteed to be the shortest path.  
but in DFS it explores a path deeply first before the exploring the next, therefore it might not find the shortest path, that is why DFS produce winding and suboptimal routes.

4. (Evaluate) If we scaled `visual\_grid\_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

**BFS & UCS:** Stores all the nodes at current level  
on 1000x1000 grid, bfs stores ~1000000 nodes, thus crashing the program due to memory exhaustion

but in **DFS:** stores only one path at a time  
on 1000x1000 grid, dfs only stores ~1000 nodes, therefore dfs is memory efficient than bfs and ucs.

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

Finalize and Lock Lab 03 Documentation

**Incomplete: 0/11 questions answered. Please provide detailed responses for all questions.**



**FACULTY OF COMPUTING**